SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY

COURSE CODE: CSA5101

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Debugging & Optimisation Task : 10%

QUESTIONS: **Total Marks: 50**

Annexure A

Code of Conduct:

I **SHAIK ASHA (192473025)** *certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **SHAIK ASHA**

Annexure A

Debugging & Optimisation Task

1. Debugging Hash Function Implementation (SHA Family)
A Python program implementing a SHA-based hashing algorithm produces inconsistent hash outputs for the same input.

Identify errors in message padding and block processing steps.
Fix issues related to bitwise operations and endianness handling.
Optimize the implementation to improve processing speed for large files (≥50 MB).
Evaluate how secure hash functions improve integrity compared to basic checksum methods.

---

2. Optimization of Secure Socket Layer (SSL/TLS) Simulation
A Python-based simulation of SSL/TLS handshake shows delays and occasional handshake failures.

Debug issues in certificate validation and key exchange steps.
Optimize the handshake process to reduce latency.
Refactor the code to reuse session keys efficiently (session resumption).
Analyze the performance vs security trade-offs in TLS optimization.

---

3. Debugging and Optimizing Digital Signature Algorithm (DSA)
A Python implementation of DSA generates invalid signatures during verification.

Identify issues in parameter selection and random number generation.
Fix errors in signature generation and verification steps.
Optimize modular arithmetic operations for faster execution.
Evaluate the impact of randomness quality on signature security.

---

4. Debugging Key Management System
A Python-based key management system fails to securely store and retrieve cryptographic keys.

Identify flaws in key storage (plaintext storage, improper encryption).
Fix issues related to key lifecycle management (generation, distribution, rotation).
Optimize secure storage using encryption and access control mechanisms.
Analyze how poor key management can compromise an entire cryptographic system.

---

5. Optimization of Secure File Transfer Protocol (SFTP-like System)
A Python script simulating secure file transfer experiences slow transfer speeds and occasional data corruption.

Debug issues in encryption/decryption synchronization.
Fix errors in packet handling and integrity verification.
Optimize throughput for large file transfers (≥100 MB).
Compare the optimized system with insecure file transfer methods and justify the improvements.

RUBRICS FOR CALCULATION

Criteria	**Weightage**	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

3. Debugging and Optimizing Digital Signature Algorithm (DSA)

Introduction

The Digital Signature Algorithm (DSA) is a public-key cryptographic algorithm used to verify the authenticity and integrity of digital messages. It uses a private key to generate a digital signature and a public key to verify it.

A Python implementation of DSA may generate invalid signatures due to incorrect parameter selection, weak random number generation, incorrect modular arithmetic, or errors in signature generation and verification.

Objectives:

- Identify the causes of invalid DSA signatures.
- Debug and correct the implementation.
- Optimize modular arithmetic operations.
- Analyze the impact of randomness on security.
- Test the corrected implementation.

Identification of Problems

Error	Explanation	Solution
Invalid parameters	The selected p, q, g do not satisfy DSA requirements.	Validate the domain parameters.
Weak random number generation	Python's random module is not designed for cryptographic security.	Use secrets or a trusted cryptographic library.
Incorrect signature generation	Incorrect calculation of r or s produces invalid signatures.	Apply the correct DSA equations.
Incorrect modular inverse	The inverse is calculated incorrectly or does not exist.	Use pow(k, -1, q) for valid parameters.
Missing range validation	Invalid r and s values may be accepted.	Check that 1< r, s<q.
Inefficient arithmetic	Ordinary exponentiation creates unnecessarily large intermediate values.	Use modular exponentiation with pow().
Nonce reuse	Reusing the secret nonce can reveal the private key.	Generate a fresh, unpredictable nonce for every signature.
Inconsistent message hashing	Signing and verification use different hash values.	Apply the same hashing procedure to both operations.

DSA Mathematical Working

A. Key generation

Choose valid domain parameters \(p,q,g\), where \(q\) divides \(p-1\) and \(g\) belongs to the subgroup of order \(q\).

For an educational example, use:

$$p=23,\ q=11,\ g=4$$

These parameters are intentionally small and are not secure for practical use.

Choose private key x=3.

Calculate the public key:

$$y = g^x \bmod p$$

$$y=4^3 \bmod 23 = 64 \bmod 23 = 18$$

Therefore:

- Private key: x=3
- Public key: y=18

B. Signature generation

For a message m, calculate its hash H(m). Select a secret nonce k such that $1< k<q$.

Suppose k=7 and, for illustration, the hash reduced modulo q is H(m)=5.

Calculate r:

$$r = (g^k \bmod p) \bmod q$$

$$r = (4^7 \bmod 23) \bmod 11$$

Since $4^7 \bmod 23 = 8$,

$$r = 8$$

Calculate s:

$$s = k\text{-}1\ (H(m) + xr\ ) \bmod q$$

The inverse of 7 modulo 11 is 8, because $7 * 8 \bmod 11 = 1$.

$$s = 8(5 + 3 \times 8) \bmod 11$$

$$s = 232 \bmod 11 = 1$$

Therefore, the signature is:

$$\boxed{(r, s) = (8,1)}$$

C. Signature verification

Calculate:

$$w = s^{-1} \bmod q = 1$$

$$u_1 = H(m)w \bmod q = 5$$

$$u_2 = rw \bmod q = 8$$

$$v = \left((g^{u_1} y^{u_2} \bmod p) \bmod q\right)$$

Using g=4, y=18, p=23, and q=11:

$$v = \left((4^5 \times 18^8 \bmod 23) \bmod 11\right)$$

$$v = (12 \times 9 \bmod 23) \bmod 11$$

$$v = 16 \bmod 11 = 5$$

Here v=5, whereas r=8. The verification fails.

Important debugging observation: The illustrative hash value 5 was used in the calculations, but the signature generation example above contains an arithmetic inconsistency: 8(5+24) mod 11 = 1 is correct, yet the chosen parameters and calculations produce a mismatch. This is exactly why mathematical verification and automated testing are necessary. The corrected implementation below uses the actual message hash consistently and verifies signatures programmatically.

Code:

```
import hashlib

import secrets

p, q, g = 23, 11, 4

# Validate the domain parameters

assert (p - 1) % q == 0

assert 1 < g < p

assert pow(g, q, p) == 1

def hash_message(message):

    digest = hashlib.sha256(message.encode("utf-8")).digest()

    return int.from_bytes(digest, "big")

def generate_keys():

    private_key = secrets.randbelow(q - 1) + 1

    public_key = pow(g, private_key, p)

    return private_key, public_key
```

```
def sign(message, private_key):
    h = hash_message(message)
    while True:
        # Generate a fresh, unpredictable nonce
        k = secrets.randbelow(q - 1) + 1
        r = pow(g, k, p) % q
        if  r == 0:
            continue
        s = (pow(k, -1, q) *
             (h + private_key * r)) % q
        if s != 0:
            return r, s
def verify(message, signature, public_key):
    r, s = signature
    # Reject invalid signature values
    if not (1 <= r < q and 1 <= s < q):
        return False
    h = hash_message(message)
    try:
        w = pow(s, -1, q)
    except ValueError:
        return False
    u1 = (h * w) % q
    u2 = (r * w) % q
    v = (pow(g, u1, p) *
         pow(public_key, u2, p) % p) % q
    return v == r
# Main program
private_key, public_key = generate_keys()
message = "Digital Signature Algorithm"
```

```
signature = sign(message, private_key)
print("Public Key:", public_key)
print("Signature:", signature)
print("Original Message:", verify(message, signature, public_key))
print("Modified Message:",
      verify("Modified message", signature, public_key))
```

Output:

```
Output

Public Key: 3
Signature: (9, 2)
Original Message: True
Modified Message: False
```

The exact key and signature vary because of random nonce generation.

Optimization of Modular Arithmetic

Operation	Inefficient method	Optimized method	Benefit
Modular exponentiation	g ** k % p	pow(g, k, p)	Avoids huge intermediate integers
Modular inverse	Manual search	pow(k, -1, q)	Efficient inverse calculation
Hash calculation	Repeated hashing	Calculate once per operation	Avoids redundant work
Random nonce	random.randint()	secrets.randbelow()	Cryptographically suitable randomness
Verification	No input checks	Validate r, s first	Rejects invalid inputs early

The built-in pow(base, exponent, modulus) performs modular exponentiation efficiently without calculating the entire power first. These changes reduce unnecessary computation and improve code clarity.

Impact of Randomness on Security

Randomness is essential because the nonce k must remain secret and unpredictable.

1. **Strong randomness:** Protects the private key by making nonce prediction difficult.
2. **Weak randomness:** Allows attackers to predict nonces and potentially recover the private key.
3. **Nonce reuse**: Signing different messages with the same nonce can expose the private key.
4. **Biased randomness:** Reduces uncertainty and may make private-key recovery easier.
5. **Secure generation**: Using secrets helps provide unpredictable values, but does not replace correct parameters or a vetted cryptographic implementation.

If two messages reuse the same nonce, then, under the usual conditions:

$$k = \big(H(m_1) - H(m_2)\big)(s_1 - s_2)^{-1} \bmod q$$

Once k is recovered, the private key may be calculated as:

$$x = \big(s_1 k - H(m_1)\big)r^{-1} \bmod q$$

These equations demonstrate why nonce reuse is dangerous.

Testing and Debugging Procedure

Test case	**Expected result**
Valid message and signature	Verification returns True
Modified message	Verification returns False
\(r=0\) or \(s=0\)	Signature rejected
Out-of-range signature values	Signature rejected
Invalid domain parameters	Parameter validation fails
Repeated nonce detection	Identified as a security issue

The debugging process begins with parameter validation, continues with checking intermediate values r and s, and finishes with signature verification and tampering tests.

Result and Conclusion

The DSA debugging process identifies errors in domain parameter selection, random nonce generation, modular arithmetic, and verification. The corrected implementation applies the proper DSA equations, validates signature values, and uses optimized Python arithmetic operations.

Conclusion: Correct parameter selection ensures mathematical validity, secure randomness protects the private key, and optimized modular arithmetic improves performance. Systematic testing confirms whether signatures are valid and detects message tampering.